
Arquitectura del Sistema — SIGAI-SES

Securitas Colombia S.A.



Arquitectura del Sistema — SIGAI-SES

VERSION	1.0.0	STATUS	STABLE	LAST UPDATE	JULIO 2026
STACK	REACT FASTAPI SQL		FRONTEND	REACT 18 / TYPESCRIPT	
BACKEND	FASTAPI / PYTHON 3.12		DATABASE	POSTGRESQL / MYSQL	

[!IMPORTANT] Este documento describe la **arquitectura global** del SIGAI-SES, incluyendo componentes, flujos de informacion, patrones de diseno y decisiones arquitectonicas clave.

1. Diagrama de Arquitectura Global

Diagrama de Arquitectura Global

Figure 1: Diagrama de Arquitectura Global

Modelo de Contenedores — Visión general del sistema

Flujo de Comunicación

```
graph TB
  subgraph "🌐 Cliente"
    A[React SPA<br/>Vite + TS]
  end
  subgraph "🌐 Servidor"
    B[Nginx<br/>Reverse Proxy]
    C[FastAPI<br/>Uvicorn Workers]
  end
  subgraph "🌐 Datos"
    D[(Base de Datos<br/>PostgreSQL / MySQL)]
  end
  A -->|HTTPS| B
```

```
B -->|proxy_pass :8000| C
C -->|asyncpg / aiomysql pool| D
```

2. Diagrama de Flujo de Información

Flujo de Informacion

Figure 2: Flujo de Informacion

Excel → Procesamiento → Dashboard

Pipeline de Datos

flowchart LR

```
A[Excel<br/>Upload] --> B[Deteccion<br/>de Tipo]
B --> C[Normalizacion<br/>y Validacion]
C --> D{Upsert}
D -->|Nuevo| E[INSERT]
D -->|Existente| F[UPDATE]
E --> G[Base de Datos]
F --> G
G --> H[Dashboard<br/>en Tiempo Real]
```

3. Descripción Detallada de Componentes

3.1 Interfaz de Usuario (Lado del Cliente)

Atributo	Detalle
Framework	React 18.2 + TypeScript 5.x
Bundler	Vite 5.2
Estilos	Tailwind CSS 3.4 — Fusion UI (Emerald Core × Neomorphic Hub)
HTTP	Axios 1.6 con interceptores JWT
Ruteo	React Router DOM — 16 rutas (1 pública, 15 protegidas)
Estado	AuthContext + TanStack Query 5

Módulos del Frontend

Modulo	Ruta	Rol Mínimo	Descripcion
Login	/login	Público	Autenticación con react-hook-form
Dashboard	/	TECNICO	KPIs, gráficos, alertas, movimientos
Inventario	/inventory	TECNICO	CRUD items/activos, import/export
Clientes	/clients	TECNICO	CRUD clientes corporativos
Proyectos	/projects	TECNICO	CRUD proyectos con clientes
Garantías	/guarantees	TECNICO	Flujo completo de garantías

Modulo	Ruta	Rol Mínimo	Descripción
Alertas	/alerts	TECNICO	Centro de alertas y gestión
Desmontes	/desmontes	TECNICO_LAB	Triage de equipos
Usuarios	/users	ADMIN	CRUD usuarios, roles, regionales
Auditoría	/audit	ADMIN	Bitácora de cambios
Entregas	/deliveries	ADMIN	Actas de entrega + PDF

[!TIP] Los módulos de **Usuarios**, **Auditoria** y **Entregas** son exclusivos para rol ADMIN.

3.2 Servidor de Aplicaciones (Lado del Servidor)

Atributo	Detalle
Framework	FastAPI 0.136 (Python 3.12)
Servidor	Uvicorn ASGI
ORM	SQLAlchemy 2.0 Async + asyncpg / aiomysql
Documentación	OpenAPI 3.0.3 (/docs, /redoc)
Seguridad	OAuth2 Password Flow + JWT + bcrypt
Estructura	10 módulos API, 7 CRUD, 1 importación

Módulos del Backend

Modulo API	Endpoints	Descripcion
/api/v1/auth	5	Login, refresh, logout, register, me
/api/v1/users	10	CRUD + audit + settings + avatar
/api/v1/inventory	17	CRUD items, activos, ubicaciones, desmonte-bulk, epp
/api/v1/business	27	CRUD clientes, proyectos, proveedores, garantías, actas
/api/v1/analytics	3	Summary dashboard, search global, predicciones
/api/v1/reports	1	Export Excel/PDF (5 módulos)
/api/v1/alerts	7	CRUD alertas + summary + evaluar
/api/v1/regionales	4	CRUD de regionales
/api/v1/import	3	Importación Excel (auto-detección) + plantillas
/api/v1/monitoring	3	Health check, health/db, metrics

3.3 Base de Datos y Almacenamiento

Atributo	Detalle
Motor	PostgreSQL 16+ / MySQL 8.0+ / MariaDB 10.5+
Pool	asyncpg / aiomysql — pool_size=30, max_overflow=50
ORM	SQLAlchemy 2.0 Async (Base declarativa)

Atributo	Detalle
Migraciones	Alembic — 14 versiones aplicadas

Tablas del Sistema (18)

Modulo	Tablas	Descripcion
Seguridad	usuarios, regionales, sesiones_usuario	Acceso y control
Inventario	items, activos, stock_bulk, movimientos_inventario, historial_ubicaciones, epp_asignaciones	Gestión de activos
Garantías	garantias	Seguimiento RMA
Entregas	actas_entrega, detalles_acta_entrega	Actas digitales
Negocio	clientes, proveedores, proyectos	Entidades de negocio
Auditoría	audit_logs	Registro inmutable
Alertas	alerts, alert_rules	Motor de reglas
Vistas	v_stock_consolidado, v_dashboard_kpis	Vistas materializadas

[!NOTE] Las **vistas materializadas** se actualizan periódicamente para optimizar consultas del dashboard.

4. Flujo de Comunicación Detallado

sequenceDiagram

participant U as ☒ Usuario
participant N as ☒ Nginx
participant F as ☒ FastAPI
participant M as ☒ Base de Datos

U->>N: HTTPS Request
N->>F: proxy_pass :8000
activate F

Note over F: ☒ Valida JWT (python-jose)
Note over F: ☒ Autoriza rol (RBAC)
Note over F: ☒ Valida datos (Pydantic)
Note over F: ☒ Ejecuta CRUD
Note over F: ☒ Registra auditoría

F->>M: Consulta SQLAlchemy Async
activate M
M-->>F: Resultado
deactivate M

F-->>N: JSON Response
deactivate F
N-->>U: HTTPS Response

Note over U: ☒ React actualiza estado
Note over U: ☒ Re-renderiza vista

5. Decisiones Arquitectónicas (ADRs)

ID	Decisión	Justificación	Alternativas
ADR-01	FastAPI sobre Django	Rendimiento async nativo, tipado con Pydantic, OpenAPI automático	Django REST (síncrono, más pesado)
ADR-02	React + Vite sobre Next.js	SPA ligera, HMR rápido, deploy simple en Vercel	Next.js (SSR complejo para PWA)

ID	Decisión	Justificación	Alternativas
ADR-03	SQLAlchemy Async sobre Tortoise ORM	Madurez, documentación, Alembic	Tortoise ORM (menos maduro)
ADR-04	JWT en sessionStorage sobre httpOnly cookies	Simplicidad, SPA sin SSR	httpOnly cookies (más seguro, pero complejo)
ADR-05	Compatibilidad multiple PostgreSQL / MySQL / MariaDB	Soporta el motor que TI defina, mediante SQLAlchemy abstracto	Dependencia de un solo motor (menos flexible)

[!WARNING] **ADR-04** (JWT en sessionStorage) es un riesgo de seguridad asumido. Mitigado con expiración de 8h y sesiones revocables (token de refresh almacenado en la tabla `sesiones_usuario`).

6. Patrones de Diseño Utilizados

Patrón	Ubicación	Propósito
Repository	<code>app/crud/*.py</code>	Abstracción de acceso a datos
Dependency Injection	<code>app/api/deps.py</code>	Inyección de dependencias (BD, usuario)
Factory	<code>app/models/*.py</code>	Creación de modelos SQLAlchemy
Singleton	<code>app/db/session.py</code>	Instancia única del engine BD
Observer	<code>app/alerts/rules.py</code>	Motor de reglas → alertas
Strategy	<code>app/services/import_service.py</code>	Estrategia según tipo de archivo

```
graph LR
  subgraph "Patrones de Diseño"
    A[Repository] --> B[Abstracción BD]
    C[DI] --> D[Inyección Seguridad]
    E[Observer] --> F[Motor Alertas]
    G[Strategy] --> H[Importación]
  end
```

7. Seguridad en la Arquitectura

Capa	Medida	Detalle
Autenticación	OAuth2 Password Flow	JWT access 8h + refresh 7d
Autorización	RBAC	<code>require_roles(["ADMIN", ...])</code>
Aislamiento	Filtro por regional	Obligatorio en consultas
Rate Limiting	SlowAPI	10 req/min en login
CORS	Restringido	Solo orígenes en .env
Auditoría	<code>audit_logs</code>	Todas las operaciones CRUD
Soft Delete	<code>deleted_at</code>	Items, clientes, proveedores
Atomicidad	Transacciones	<code>session.commit()</code>

[!NOTE] El **filtro por regional** garantiza que un usuario solo vea datos de su región asignada, implementando aislamiento a nivel de datos.

-DOCUMENTO ACTUALIZADO AL JULIO 2026 - V1.0.0

Figure 3: Separator

[!IMPORTANT] Para preguntas sobre la arquitectura, contacte al **equipo de arquitectura** en [#sigai-ses-arch](#) o revise los ADRs completos en [docs/adr/](#).